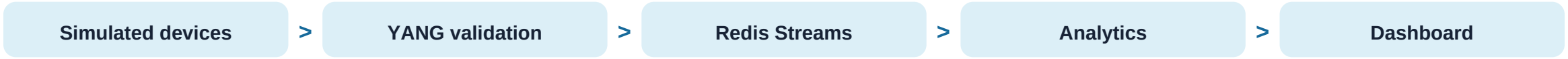
PROJECT OVERVIEW

Schema-Validated YANG-Push Telemetry Pipeline

A reliable, standards-aligned telemetry pipeline for trustworthy network monitoring and explainable anomaly detection.

- Network devices continuously report operational information, including interface status, traffic counters and performance statistics.
- The proposed system receives simulated YANG-Push notifications and checks each message against an actual YANG data model before downstream processing.
- Malformed or schema-invalid notifications are quarantined with a diagnostic reason. Valid notifications continue through Redis Streams for processing and storage.
- An analytics layer identifies unusual but schema-valid behaviour using adaptive statistical methods and supplies numerical evidence to the dashboard.
- Implementation is scoped as a modular monolith for two team members, rather than a large distributed platform.



PROBLEM DESCRIPTION

Why Schema Validation Matters

Streaming telemetry is useful only when receivers can trust the structure and meaning of incoming data.

- High-frequency telemetry may contain malformed JSON, unexpected data-tree nodes, wrong data types or impossible values.
- A payload can be syntactically valid JSON and still violate the schema that defines its expected fields, types, ranges or relationships.
- If such data enters a broker and database unchecked, it can distort traffic calculations, trigger false alerts and reduce confidence in the monitoring system.
- Duplicate or delayed notifications can also distort counts and time-based analysis; consumer crashes can leave data unprocessed.
- The central challenge is to build a validation boundary before downstream processing while retaining diagnostic information and measuring reliability.

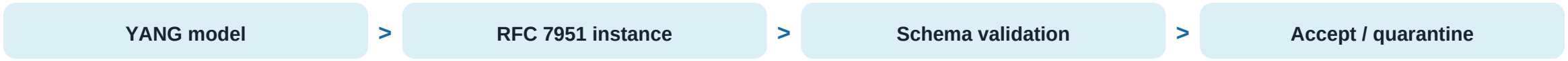
What our pipeline must prove

- Invalid notifications are rejected and identifiable by failure reason, while valid messages are forwarded.
- Notification structure is preserved through the message broker; loss, duplicates, delay and restart behaviour are measurable.

YANG, YANG-Push and the Data Contract

YANG is the blueprint for the data; YANG-Push is the subscription-based delivery mechanism.

- YANG defines a tree of containers, lists and leaves together with their permitted data types, ranges, enumerations and model constraints.
- RFC 7950 defines YANG 1.1. RFC 7951 defines the JSON encoding for data modeled using YANG.
- RFC 8641 describes YANG-Push notifications, including periodic push-update and on-change push-change-update concepts.
- Our focused MVP uses ietf-interfaces.yang as the primary designated model. A small custom-system-telemetry module is an optional extension for CPU, memory and latency.
- An RFC 8525 YANG Library metadata file supplies schema context. The yangson library is proposed to load the modules and validate instance data.
- HTTP POST is the emulator transport for the hackathon. It should not be presented as a native NETCONF hardware subscription.

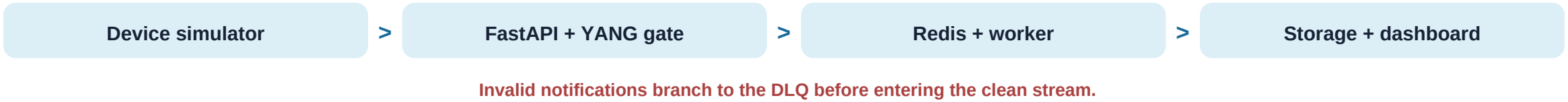


PROPOSED SOLUTION

End-to-End Architecture

A single Python application coordinates validation, streaming, analytics and presentation, supported by Redis and TimescaleDB.

- Four simulated network devices generate periodic telemetry and controllable faults, allowing reproducible demonstrations without physical hardware.
- FastAPI receives notifications at an HTTP endpoint. The gateway identifies duplicates and invokes the schema-validation pipeline.
- Accepted data enters Redis Streams. A consumer reads it, normalizes metrics, writes records to TimescaleDB and updates analytics.
- Invalid data follows a separate Dead-Letter Queue path with failure stage, schema path where available, error reason and a payload reference.
- Processed telemetry, device status and anomaly evidence are exposed through APIs and WebSocket updates to a React dashboard.
- Three proposed containers: telemetry_app, telemetry_redis and telemetry_timescaledb.



Invalid notifications branch to the DLQ before entering the clean stream.

VALIDATION DESIGN

The Six-Stage YANG Validation Pipeline

Schema-aware checks are the core engineering feature, not a superficial JSON type checker.

Stages 1-4

- 1. Syntax: parse UTF-8 JSON and reject malformed input before inspecting the model.
- 2. Envelope: verify the expected notification wrapper, event time and YANG-Push update structure.
- 3. Schema resolution: locate the YANG module and revision required to interpret the instance.
- 4. Structure: check containers, lists, keys, expected hierarchy and unexpected nodes.

Stages 5-6 and outcomes

- 5. Type and range: validate the schema-defined types, enumerations, numeric ranges and other supported constraints.
- 6. Semantic constraints: demonstrate at least one must expression defined in the model and evaluated by the YANG-aware validator.
- On failure: record the stage, error code, relevant model path and reason; retain the rejected payload for examination.
- On success: produce a structured valid event for downstream ingestion and normalization.

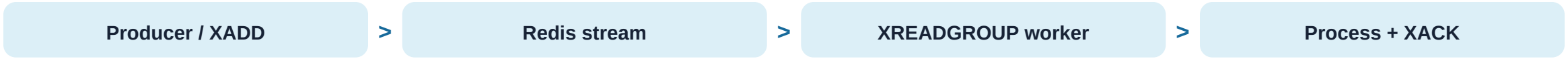
Illustrative semantic rule: reject a telemetry instance when an explicitly modeled counter relationship is violated. The precise constraint must be present in the selected .yang model.

STREAM RELIABILITY

Redis Streams, Deduplication and Recovery

A reliable pipeline must behave predictably when events repeat or a consumer is interrupted.

- XADD appends each accepted event to the telemetry stream, while XREADGROUP reads messages through a named consumer group.
- XACK acknowledges a message only after the required processing has completed. XPENDING reveals messages delivered but not yet acknowledged.
- During a consumer restart test, inspect pending messages and recover or claim outstanding work before marking it complete.
- Deduplication uses a deterministic SHA-256 fingerprint based on device identity, subscription, event time and canonicalized message content; a time-limited Redis key can identify repeats.
- Bounded retention is configured for the hackathon to prevent uncontrolled in-memory growth, while the risk of trimming unprocessed entries is explicitly measured or controlled.
- The pipeline tracks unique accepted events, duplicate detections and message-processing outcomes separately.



Adaptive Anomaly Detection with Evidence

Schema-valid telemetry is not automatically normal: analytics identifies unusual conditions only after validation.

- EWMA maintains an adaptive baseline that reflects recent observations and highlights departures from expected conditions.
- MAD and robust Z-scores quantify deviation with reduced sensitivity to extreme observations compared with mean/std-based scoring.
- CUSUM accumulates smaller sustained departures and can detect gradual or persistent operational changes.
- Isolation Forest is an optional secondary multivariate detector for combinations of CPU, latency and interface metrics.
- Alerts should contain observed value, baseline, score, triggering detectors and severity criteria rather than a label without explanation.
- Per-device baselines are preferable to assuming that every router, switch or gateway has the same operating profile.

Illustrative example - not an experimental result

- A device with a recent CPU baseline near 45% reports 91%. The analytics layer evaluates the deviation, records evidence and shows an alert if the configured decision criteria are met.

Controlled Fault-Injection Scenarios

The simulator makes both expected successes and expected failures observable to judges.

Normal notification	Accepted; reaches broker and dashboard
Malformed JSON	Rejected at syntax check; recorded in DLQ
Wrong datatype	Rejected against the YANG-defined leaf type
Out-of-range value	Rejected against the model constraint
Semantic violation	Rejected by the implemented must constraint
CPU spike or gradual drift	Valid telemetry; evaluated by anomaly detectors
Repeated notification	Duplicate identified without double counting
Consumer restart	Pending-message recovery behaviour measured

Each run should record its injected condition, expected result, observed outcome and any discrepancy.

EVALUATION METHODOLOGY

What We Will Measure

Numerical performance and reliability claims must come from actual controlled test runs.

Message loss / accountability	Compare messages sent with valid accepted, rejected, pending and successfully consumed counts; do not call intentional rejection a loss.
Duplicate detection	Inject known duplicate events and compare injected versus detected/suppressed duplicates.
Delivery delay	Record event and processing timestamps; report median and p95/p99 when sample size supports it.
Restart recovery	Stop a worker, inspect pending entries, restart/recover and count eventually acknowledged messages.
Structure preservation	Compare required YANG-Push notification fields before and after broker serialization and consumption.
Invalid capture rate	Injected invalid notifications successfully quarantined divided by the total injected invalid notifications.

All metrics are planned measurements. No achieved throughput, latency or F1 score is claimed at proposal stage.

IMPLEMENTATION PLAN

Two Members, Clear Responsibilities

A modular monolith and shared event contract reduce integration overhead while preserving technical depth.

Person A - Protocol and validation

- Prepare the YANG model(s), dependencies and schema-library metadata.
- Implement YANG-aware schema loading, validation stages and structured diagnostics.
- Build the simulator, malformed cases and fault-injection controls.
- Test validation correctness and rejected-message handling.

Person B - Systems, analytics and interface

- Set up Docker, Redis Streams, TimescaleDB and FastAPI processing.
- Implement consumer groups, deduplication, restart and storage paths.
- Build statistical detectors and evidence-producing anomaly records.
- Develop the React dashboard and reliability measurement instrumentation.

Shared integration: a common Pydantic event contract, repeatable end-to-end tests, demonstrated fault scenarios, repository documentation and judge-ready walkthrough.

Validate. Quarantine. Stream. Analyze. Explain.

The project prioritizes trustworthy, inspectable network telemetry over a visually impressive but unverified dashboard.

- Schema-aware acceptance: actual YANG models govern the trusted data boundary.
- Diagnosable rejection: invalid messages are preserved with actionable errors.
- Broker-backed processing: Redis Streams makes acknowledgements and recovery behaviour inspectable.
- Explainable monitoring: statistical evidence accompanies detected anomalies.
- Measurable engineering: loss, duplicates, delivery delay, restart recovery and structure preservation are evaluated.
- Focused scope: simulated devices and HTTP ingestion; native physical-router NETCONF/gNMI and full vendor interoperability are future extensions.

Key references

RFC 7950: YANG 1.1 | RFC 7951: JSON Encoding of YANG Data

RFC 8641: YANG-Push | RFC 8525: YANG Library

Redis Streams / Consumer Groups documentation | Official AIORI PS-034 statement